

Grupo 10

Reentrega - Informe Diseño de Compiladores

Etapas 3 y 4: Generación de Código Intermedio y Generación de Código de Máquina

Universidad Nacional del Centro de la Provincia de Buenos Aires.

Alumnos

De Cáceres, Gonzalo

gonzadecaseres@gmail.com

Halty, Héctor Manuel

hectormanuelhalty@gmail.com

Lasarte, Fermin

fermin.lasarte@icloud.com

Tutor

José Fernández León

Temas Particulares Asignados

2 5 8 9 14 19 21 23 25 28 31 32 - b e f

Introducción

El presente informe constituye la reentrega de las etapas finales del proyecto de Diseño de Compiladores, abarcando específicamente la Generación de Código Intermedio (Etapa 3) y la Generación de Código de Máquina (Etapa 4). El objetivo central de esta revisión ha sido consolidar la arquitectura del compilador para garantizar no solo la traducción correcta de las estructuras sintácticas a Tercetos y posteriormente a Assembler 8086, sino también asegurar la robustez del ejecutable final.

El diseño mantiene el soporte para los temas particulares asignados al Grupo 10, que incluyen el manejo de tipos de datos específicos (uint de 16 bits y float de 32 bits), estructuras de control iterativas (do-while), asignaciones múltiples, funciones con paso de parámetros por copia-resultado, y construcciones complejas como las expresiones lambda. Además, se incluyen los controles estrictos en tiempo de ejecución para situaciones de error aritmético (overflow y resultados negativos en enteros sin signo).

Sin embargo, el diferencial fundamental de esta entrega radica en la metodología de validación aplicada. A diferencia de la iteración anterior, en esta instancia se ha desarrollado y ejecutado una batería de casos de prueba extensivos y rigurosos. Este enfoque exhaustivo en el testing permitió someter al compilador a escenarios de borde y combinaciones complejas de sentencias, con el fin de detectar inconsistencias que previamente pasaron desapercibidas. A lo largo de este documento, se detallan las decisiones de diseño tomadas y cómo estas correcciones garantizan que el producto final cumpla con todas las especificaciones funcionales requeridas.

Modificaciones a las etapas anteriores

Para esta reentrega, se han implementado diversas modificaciones orientadas a mejorar la robustez del compilador y asegurar el cumplimiento de los temas particulares asignados.

Modificaciones en el Análisis Léxico y Sintáctico

- **Recuperación en Bloques IF:** Implementación de reglas para inferir el cierre de bloques condicionales ante la ausencia de la palabra reservada ENDIF.
- **Corrección de Operadores:** Mecanismo de inferencia automática para corregir errores tipográficos en el operador de asignación (:=).
- **Validación de RETURN:** Incorporación de controles semánticos para asegurar la existencia de sentencias de retorno en todas las funciones declaradas.
- **Inferencia de Fin de Sentencia:** Flexibilización de la gramática para tolerar y advertir sobre la ausencia de puntos y coma al final de las instrucciones.
- **Visualización de Tabla de Símbolos:** Actualización del módulo de reporte para incluir la columna "Pasaje" en la impresión de la Tabla de Símbolos, permitiendo verificar el tipo de pasaje de parámetros (copia-valor, copia-resultado, etc.) asignado a cada identificador. Tipo (para inferencia y chequeos), Uso (para distinguir variables de funciones), y listas de Parámetros y TiposRetorno para soportar la sobrecarga y validaciones de invocación.

Manejo de Tipos y Controles de Ejecución

- **Gestión de UINT:** Implementación de corrección de literales negativos en tiempo de compilación y controles de desbordamiento (overflow) en tiempo de ejecución.
- **Saturación de FLOAT:** Verificación de rangos y control estricto de overflow, reportando error explícito ante desbordamientos.

Estructuras de Control y Lambdas

- **Recuperación en DO-WHILE:** Incorporación de soporte para manejar condiciones inválidas o mal formadas dentro de las estructuras de iteración.
- **Manejo de Errores en Lambdas:** Adición de reglas para permitir el análisis continuo del cuerpo de expresiones Lambda, incluso si su definición presenta errores.
- **Restricción de Declaraciones:** Implementación de validaciones semánticas para impedir declaraciones de variables dentro de bloques de código ejecutable.

Generación de Código Intermedio

La lógica central de esta etapa reside en la clase `Generador.java`. Esta clase implementa el patrón de diseño Singleton, lo que garantiza la existencia de una instancia única que centraliza la creación de tercetos y la gestión de las estructuras auxiliares (pilas de operandos, control y parámetros). Esta instancia es accesible globalmente desde las acciones semánticas del analizador sintáctico definido en `gramatica.y`. Además, el Generador actúa como coordinador con la Tabla de Símbolos para la resolución de nombres (Name Mangling) y la validación de tipos durante la creación de las instrucciones intermedias.

Información agregada a la TS

Para soportar las validaciones semánticas y la generación de código intermedio de manera robusta, la Tabla de Símbolos se enriqueció con metadatos esenciales que trascienden el simple lexema. Se destaca la incorporación explícita del atributo Tipo desde la etapa de Análisis Léxico para las constantes (uint, float), lo que elimina la necesidad de re-inferir tipos basados en cadenas en etapas posteriores. Sintácticamente, se determina el tipo para identificadores, soportando la inferencia de tipos (Tema 9) y los retornos de función. El atributo Uso distingue el rol semántico de cada entrada (variable, funcion, parametro, parametro_lambda, nombre_programa). Además, para cumplir con los requerimientos complejos, se gestionan atributos estructurados como RetornoMultiple y TiposRetorno (Tema 21), y listas de Parametros. Como mejora en esta versión, el atributo Pasaje no solo controla la semántica de Copia-Resultado (Tema 25), sino que ahora se visualiza explícitamente en el reporte de la Tabla de Símbolos, facilitando la verificación del diseño. Todo ello se organiza mediante Name Mangling para manejar correctamente el alcance y el ocultamiento de variables en distintos ámbitos.

Notación Posicional en YACC

Para la gestión de los atributos semánticos asociados a los símbolos de la gramática, se empleó la notación posicional característica de YACC. Esta sintaxis permite acceder a los valores de los tokens y no terminales dentro de una regla de producción, facilitando la propagación de información a través del árbol de análisis sintáctico.

La notación se estructura de la siguiente manera:

- **\$\$**: Representa el valor semántico del símbolo no terminal situado en el lado izquierdo de la regla (el resultado de la reducción).
- **\$n**: Representa el valor semántico del componente en la *n*-ésima posición del lado derecho de la regla.

En la implementación del compilador, estos valores corresponden a instancias de la clase `ParserVal`. Específicamente, se utilizó el campo `.sval` (de tipo `String`) para almacenar y recuperar los lexemas identificados. Esta mecánica fue imprescindible durante la generación de código intermedio, permitiendo recuperar las claves de la Tabla de Símbolos de los operandos para la construcción automatizada de los tercetos. El ejemplo presentado a continuación es ideal para representar el uso de la notación posicional, ya que muestra claramente cómo se toman dos valores hijos (`$1` y `$3`), se procesan (concatenación) y se asignan al padre (`$$`):

```
variable : ID PUNTO ID
        {
            $$.$sval = $1.sval + "." + $3.sval;
            $$.$ival = $1.$ival;
        }
        |
        ID
        {
            $$.$sval = $1.sval;
            $$.$ival = $1.$ival;
        }
        ;
```

En este fragmento, la regla gramatical define que una variable puede estar compuesta por un identificador (`$1`), seguido de un punto (posición 2, ignorada semánticamente), y otro identificador (`$3`). La acción semántica utiliza `$1.sval` y `$3.sval` para recuperar los lexemas de los identificadores y los concatena para formar el nombre cualificado (mangled), asignando el resultado a `$$.$sval` para que esté disponible en niveles superiores de la gramática.

```
public ParserVal(int val) { ival=val; }

/**
 * Initialize me as a double
 */
public ParserVal(double val) { dval=val; }

/**
 * Initialize me as a string
 */
public ParserVal(String val) { sval=val; }
```

Como podemos observar en la imagen, el tipo de dato asociado a la variable de valor del parser, encapsulada en la estructura `ParserVal`, se infiere directamente a partir del identificador de la variable utilizado en su definición. Esta convención de nomenclatura es fundamental para el sistema, ya que permite determinar inequívocamente cómo debe interpretarse el valor almacenado.

Específicamente, la regla de tipificación se basa en la primera letra del nombre de la variable de miembro dentro de `ParserVal`:

1. **sval**: Si el nombre de la variable comienza con la letra 's', como en `sval`, el tipo de dato se establece como `string` (cadena de texto). Esto es apropiado para almacenar secuencias de caracteres, identificadores, literales de texto, o cualquier información que deba tratarse como una cadena alfanumérica.
2. **ival**: Si la variable comienza con 'i', como en `ival`, el tipo de dato es un entero (`integer`). Este tipo es ideal para representar números sin componentes fraccionarios, como contadores, índices, o valores numéricos discretos.
3. **dval**: Si la variable comienza con 'd', como en `dval`, el tipo de dato se interpreta como `double` (doble precisión). Este es un tipo de punto flotante utilizado para almacenar números reales que requieren alta precisión, esenciales en cálculos matemáticos, científicos, o financieros.

Esta técnica de codificación del tipo de dato en el nombre de la variable interna de `ParserVal` es una característica de diseño que simplifica la gestión de tipos dentro del parser, haciendo explícito el tipo de dato esperado para cada valor retornado por las reglas de gramática.

Tercetos

Los tercetos constituyen una estructura de representación intermedia de código organizada en tres componentes. El primer campo define la operación a realizar, mientras que el segundo y el tercero actúan como operandos, los cuales pueden ser lexemas o referencias a otros tercetos. En el caso de operaciones unarias, el tercer componente puede permanecer nulo.

Cada objeto Terceto se compone de tres atributos fundamentales:

- **Operador:** Una cadena que indica la acción a realizar (por ejemplo, +, :=, BF para bifurcaciones falsas, o CALL para invocaciones).
- **Operandos:** Dos campos (operando1 y operando2) que pueden almacenar lexemas de variables, constantes, o referencias a índices de otros tercetos previamente generados (ej. [10]).
- **Tipo:** Un atributo esencial para el chequeo semántico posterior, que almacena el tipo de dato resultante de la operación (ej. uint, float).

Visualmente el terceto tiene la siguiente forma:

T1(+, 1, 2)

T2(:=, A, [T1])

Para el caso de un tercer componente nulo:

T1(print, &hola&, -)

```
public class Terceto { 12 usages  & Fermin Lasarte *
    private String operador; 6 usages
    private String operando1; 7 usages
    private String operando2; 7 usages
    private String tipo; 5 usages

    // Constructor para operaciones binarias (ej: +, -, :=)
    public Terceto(String operador, String operando1, String operando2) {
        this.operador = operador;
        this.operando1 = operando1;
        this.operando2 = operando2;
        this.tipo = "void";
    }

    // Constructor para operaciones unarias (ej: toui, RETURN)
    public Terceto(String operador, String operando1) { 1 usage  & Fermin Lasarte
        this.operador = operador;
        this.operando1 = operando1;
        this.operando2 = null;
        this.tipo = "void";
    }

    // Constructor para saltos o etiquetas (ej: BI, LABEL)
    public Terceto(String operador) { 1 usage  & Fermin Lasarte
        this.operador = operador;
        this.operando1 = null;
        this.operando2 = null;
        this.tipo = "void";
    }

    @Override  & Fermin Lasarte
    public String toString() {
        // Muestra '.' para operandos nulos, facilitando la lectura
        String op1 = (operando1 != null) ? operando1 : ".";
        String op2 = (operando2 != null) ? operando2 : ".";
        return "(" + operador + ", " + op1 + ", " + op2 + ")";
    }
}
```

Generación de Tercetos

La generación de tercetos ocurre simultáneamente con el análisis sintáctico ascendente, orquestada por la clase Generador. Para gestionar la precedencia de operadores, el anidamiento de estructuras y la recuperación ante fallos, se emplean pilas auxiliares especializadas:

1. **Gestión de Operandos (pilaOperandos):** Esta pila de tipo `Stack<String>` almacena temporalmente lexemas o referencias a los tercetos creados. En la versión actual, su comportamiento se ha robustecido: si la generación de código está inhabilitada por un error sintáctico previo, el generador apila referencias "dummy" (`[-1]`) o marcadores de error (`error_tipo`) en lugar de índices reales. Esto permite que las reglas gramaticales superiores sigan encontrando operandos en la pila y continúen el análisis semántico sin interrupciones ni excepciones de "pila vacía".
2. **Control de Flujo (pilaControl):** Utilizada primordialmente para la técnica de backpatching o "rellenado hacia atrás". Esta pila `Stack<Integer>` almacena los índices de los tercetos de salto incompleto (como BI o BT) en estructuras como IF o DO-WHILE. Una mejora clave en esta entrega es su integración con las rutinas de recuperación: si el parser infiere un cierre de bloque faltante (ej. falta ENDIF), utiliza esta pila para resolver y completar los saltos pendientes automáticamente, garantizando que la estructura de control resultante sea coherente.
3. **Algoritmo de Bifurcaciones (Pseudocódigo):** A continuación se detalla la lógica de backpatching utilizada para la sentencia IF y WHILE mediante la pilaControl:

Para sentencia IF (Condicion) Sentencia:

1. Analizar Condicion.
2. Generar terceto incompleto de Salto Condicional (BF, Condicion, ?).
3. Apilar el índice del terceto BF en pilaControl.
4. Analizar el bloque Sentencia (cuerpo del IF).
5. Al finalizar el cuerpo:
 - a. Desapilar el índice del tope de pilaControl (referencia al BF).
 - b. Completar el terceto BF apuntando al terceto actual (etiqueta de fin).

Para sentencia DO Bloque WHILE (Condicion):

1. Al inicio del DO: Guardar el índice del terceto actual (InicioBucle).
2. Analizar Bloque.
3. Analizar Condicion.
4. Generar terceto de Salto Condicional Inverso (BT o BF según corresponda)


```

public class Generador { 13 usages  & Fermin Lasarte +1

    private static volatile Generador instance; 3 usages
    private ArrayList<Terceto> tercetos; 14 usages
    private Stack<String> pilaOperandos; 4 usages
    private Stack<Integer> pilaControl; 4 usages
    private Stack<ParametroInfo> pilaParametros; 4 usages
    private Stack<ParametroRealInfo> pilaParametrosReales; 6 usages
    private Stack<String> pilaLadoDerecho; 4 usages
    private AnalizadorLexico al; 14 usages

    private boolean generacionHabilitada = true; 2 usages
    private final Terceto dummyTerceto = new Terceto( operador: "DUMMY", operando1: "_", operando2: "_");

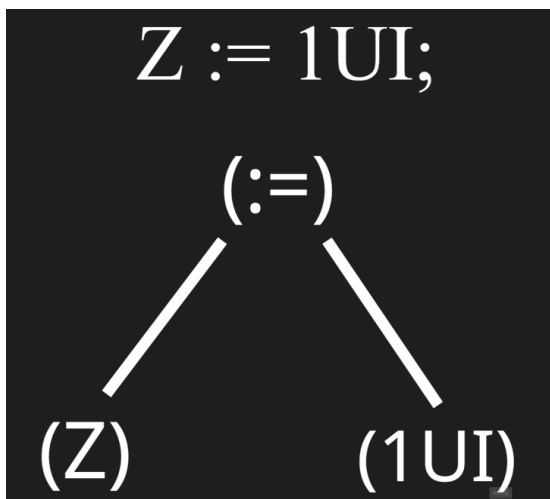
    private Generador() { 1 usage  & Fermin Lasarte
        this.tercetos = new ArrayList<Terceto>();
        this.pilaOperandos = new Stack<>();
        this.pilaControl = new Stack<>();
        this.pilaParametros = new Stack<>();
        this.pilaParametrosReales = new Stack<>();
        this.pilaLadoDerecho = new Stack<>();
    }
}

```

Generación de Tercetos a partir de Árbol Sintáctico

El proceso de generación de código intermedio implica una transformación desde la estructura jerárquica implícita en la gramática hacia una representación lineal de instrucciones. Conceptualmente, esto equivale a realizar un recorrido postorden del árbol sintáctico: se resuelven primero los nodos hijos (operandos y sub-expresiones) antes de procesar el nodo padre (operador). Dado que nuestro compilador utiliza una estrategia dirigida por la sintaxis, este "recorrido" ocurre de manera natural durante las reducciones ascendentes del parser. Esto garantiza que, al alcanzarse una regla de producción (como una asignación o una suma), los operandos necesarios ya han sido procesados y sus referencias (o índices de tercetos previos) están disponibles en la pila de operandos (pilaOperandos), gestionada por la clase Generador

Para ilustrar esta lógica, consideremos la sentencia de asignación simple `Z := 1UI;`. Estructuralmente, esta instrucción se puede representar mediante el siguiente árbol sintáctico, donde la raíz denota la operación y las hojas los operandos:



Durante el análisis, el compilador procesa primero las hojas del árbol. Al identificar la constante `1UI` (lado derecho) y la variable `Z` (lado izquierdo), se valida su información en la Tabla de Símbolos. Posteriormente, la reducción gramatical asciende hacia el nodo raíz que representa la operación de asignación `:=`. En la versión actual del compilador, este paso es robusto: si alguno de los nodos hijos contiene un error (producto de la recuperación de errores léxicos o sintácticos), el Generador no detiene el proceso, sino que propaga un estado de error controlado (`error_tipo` o terceto dummy), permitiendo que la instrucción se linealice de todas formas como `(:=, Z, 1UI)` o se descarte de manera segura. Esta linealización es fundamental, ya que descompone expresiones complejas (o definiciones anidadas como las Lambdas) en pasos discretos secuenciales, facilitando una traducción directa y tolerante a fallos hacia el código Assembler.

Temas Particulares Asignados

A continuación, se detalla la resolución de los temas específicos asignados al grupo, describiendo su tratamiento a través de las distintas etapas del compilador e incluyendo las mejoras de robustez introducidas en esta reentrega.

Tema 2: Constantes Enteras sin Signo (UINT)

- **Análisis Léxico:** La lógica principal reside en la clase interna `AccionSemantica4` dentro de `AccionSemantica.java`.
 - **Validación de Sufijo:** Se verifica explícitamente que el lexema termine con el sufijo "UI". Si falta, se reporta un error de constante mal formada.
 - **Verificación de Rango:** Para validar el rango de 16 bits (0 a 65535) sin desbordamientos durante la compilación, se utiliza la clase `BigDecimal`. Se compara el valor numérico contra el límite superior (65536).
 - **Manejo de Errores:** Si el valor excede el límite (≥ 65536), la acción semántica retorna explícitamente el token "ERROR" y emite un mensaje detallado indicando el valor que causó el fallo, impidiendo que tokens inválidos lleguen a la etapa sintáctica.
 - **Mejora Reentrega:** Se modificó la `AccionSemantica4`. Si el valor está fuera de rango, ahora se retorna explícitamente un token de ERROR, deteniendo la ejecución para evitar que el compilador trabaje con datos basura. Además, el error que se retorna es específico, aclarando que el rango permitido fue superado:
"Linea: 3 - Columna: 13 - Error Lexico: Constante uint fuera del rango permitido (0 a 65535): 70000"
- **Análisis Sintáctico:** Se modificó la gramática para interceptar literales negativos en la regla constante: `'-'` CTE..
 - **Detección de literales negativos.** Si aparece un signo menos (-) seguido de una constante UI, se emite un Warning, se trunca el signo y se toma el valor absoluto para continuar la compilación.
"Linea 3: Warning. El tipo 'uint' no admite valores negativos. Se elimino el signo menos y se usa: 1UI"
 - **Recuperación:** En lugar de detener la compilación, se elimina el signo menos y se propaga únicamente la parte positiva (`lexemaPositivo`) hacia las siguientes etapas, permitiendo generar código funcional con el valor absoluto.
- **Generación de Código:**
 - En caso de detectarse un error de overflow, no se genera ni código assembler ni código intermedio ya que es un error fatal y el compilador entra en modo pánico.
 - En caso de detectarse una constante entera negativa, no se tiene en cuenta el signo menos (-) y se trabaja con su valor absoluto. Se lanza un

Warning notificando la modificación por borrado y se genera código intermedio y código assembler normalmente.

- En la etapa de Código Intermedio, las constantes enteras sin signo se almacenan en los tercetos conservando su sufijo característico UI (ej. (+, A, 50UI)). Esto permite que el Generador identifique inequívocamente el tipo uint para realizar las validaciones de compatibilidad en operaciones mixtas. Posteriormente, durante la Generación de Assembler, el sufijo es eliminado mediante la función resolveOperand y el valor se trata como un literal inmediato de 32 bits. Sin embargo, para respetar la semántica del lenguaje (16 bits), el compilador inyecta instrucciones de verificación (CMP EAX, 65535) inmediatamente después de cada operación aritmética, asegurando que el valor resultante nunca exceda el rango permitido antes de continuar la ejecución.

Tema 5: Punto Flotante (FLOAT) y Tema 31: Controles (Overflow)

- **Análisis Léxico:** La lógica de validación se centraliza en la clase AccionSemantica6.
 - **Conversión y Precisión:** Se utiliza la clase BigDecimal para analizar el lexema con alta precisión antes de convertirlo al formato nativo. Esto permite manejar la notación científica (ej. E-38) correctamente.
 - **Validación de Rangos (Reentrega):** Se definieron límites estrictos para evitar valores basura:
 - Límite Inferior: 1.17549435E-38 (aprox).
 - Límite Superior: 3.40282347E+38 (aprox).
 - **Detección de Errores:** Si el valor leído (distinto de cero) cae fuera de este rango (Underflow o Overflow), la acción semántica retorna inmediatamente un token "ERROR" con un mensaje descriptivo (ej. "Constante flotante fuera de rango (overflow)"), bloqueando el paso de valores inválidos al analizador sintáctico.
- **Parser:** En este caso contempla la posibilidad de las constantes punto flotante negativas, modificándose su valor en la Tabla de Símbolos
 - El parser maneja los literales negativos de forma distinta a los enteros en la regla constante : '-' CTE.
 - **Actualización en Tabla de Símbolos:** Al detectar un signo menos precediendo a una constante float, no se descarta el signo. En su lugar, se invoca al.reemplazarEnTS(lexemaPositivo, lexemaNegativo), actualizando la entrada en la Tabla de Símbolos para que el lexema asociado sea el valor negativo real (ej. "-2.5"). Esto asegura que el Generador de Código recupere el valor correcto directamente.
- **Generación de Código:** Todas las operaciones aritméticas se traducen a instrucciones del coprocesador matemático (FPU) (FLD, FADD, FSTP). Se

incluye lógica para limpiar la pila del coprocesador en caso de errores de división por cero.

Tema 8: Cadenas Multilínea

- **Análisis Léxico:** El reconocimiento se apoya en la clase `AccionSemantica7`, diseñada para acumular caracteres dentro de la cadena.
 - **Acumulación:** A diferencia de las cadenas tradicionales de una sola línea, esta acción permite que el lexema crezca incluyendo caracteres de control como el salto de línea (`\n`), retornando el token `CADENA_MULTILINEA` mientras no se encuentre el delimitador de cierre.
 - **Validación de Cierre (Mejora Reentrega):** Se incorporó una validación explícita para el fin de archivo (EOF). Si el analizador léxico alcanza el final de la entrada estando aún en el estado de "lectura de cadena" (sin haber encontrado el `&` de cierre), se aborta el proceso retornando un error léxico específico ("Cadena multilinea no cerrada"), en lugar de permitir que el error se propague como un fallo sintáctico genérico. Por motivos prácticos se tomó la decisión de diseño de lanzar el error en el inicio de la cadena multilinea, y no en el final de ella.
"Linea 3: Error Lexico: Cadena multilinea no cerrada. Se esperaba el delimitador de cierre '&'."
- **Generación de Código:** Se declaran en la sección `.data` del Assembler. El generador se encarga de transformar los saltos de línea escritos en el código fuente al formato adecuado para que se visualicen correctamente en la consola, agregando además un marcador especial (byte nulo) al final para indicar dónde termina el texto.

Tema 9: Inferencia de Tipos (VAR) y Tema 23: Prefijado Opcional

- **Análisis Semántico:** La implementación de estas funcionalidades se centra en la gestión dinámica de la Tabla de Símbolos y la resolución jerárquica de nombres durante la generación de código intermedio. La resolución es puramente semántica. El tipo se infiere durante el parsing y se actualiza en la Tabla de Símbolos.
 - **Detección y Asignación:** En la regla `declaracion_var`, al procesar una sentencia como `var x := expresion`, el compilador recupera el tipo de la expresión derecha mediante `g.getTipo(expr)`.
 - **Validación:** Si el tipo es válido (no es `error_tipo`, ni `indefinido`), se actualiza inmediatamente la entrada de la variable en la Tabla de Símbolos asignándole el atributo "Tipo" inferido. Esto permite que futuras referencias a la variable ya conozcan su tipo sin necesidad de declaración explícita.
- **Resolución de Ámbitos (Prefijado):** Se utiliza la técnica de Name Mangling implementada en el `AnalizadorLexico`. Las variables se almacenan concatenando su nombre con su ámbito (ej. `VAR:MAIN`). El Generador de

Código intenta resolver referencias buscando primero en el ámbito local y escalando jerárquicamente, permitiendo el acceso a variables globales o prefijadas explícitamente por el usuario.

- **Mejora Reentrega:** Se detectó y corrigió un defecto crítico en la comunicación entre el parser y la generación de tercetos respecto a la visibilidad de variables.
 - **El Problema Anterior:** Si no se utilizaba el prefijo explícito, el Generador a veces no lograba vincular el identificador simple con su versión "mangled" en la tabla de símbolos, generando código para variables inexistentes o indefinidas.
 - **La Solución (resolveName):** Se implementó el método privado resolveName dentro de la clase Generador. Ahora, antes de crear cualquier terceto (addTerceto), el compilador invoca este método que delega en el AnalizadorLexico la tarea de encontrar el nombre único correcto (getNombreMangled).
Esto garantiza que una referencia a x en el código fuente se traduzca automáticamente a x:main o x:bloque_1 en el código intermedio según corresponda.
Esta lógica es transparente para el resto del compilador y asegura que el Assembler reciba siempre identificadores únicos y válidos.

Tema 14: Sentencia DO-WHILE

- **Análisis Sintáctico:** Se reconoce la estructura "DO" <bloque> "WHILE" (condicion).
 - **Mejora Reentrega:** Se implementó una estrategia de recuperación mediante producciones de error específicas. Se agregaron reglas gramaticales alternativas para capturar defectos estructurales comunes, como la falta de paréntesis rodeando la condición o paréntesis vacíos (). Al detectar estas estructuras erróneas, el parser ejecuta acciones semánticas de limpieza que invocan a g.desapilarControl(). Esto es crítico porque el token DO ya había apilado una etiqueta de inicio de bucle; si no se desapila manualmente ante un error en la condición, esa etiqueta quedaría "huérfana" en la pila, corrompiendo la generación de saltos de las estructuras siguientes. De esta forma, se valida el cuerpo del bucle y se neutraliza el error de la cabecera sin detener la compilación.
 - **Manejo de Errores en el Cuerpo (Modo Pánico):** A diferencia de los errores recuperables en la condición, si se detecta un error léxico, sintáctico o semántico dentro del cuerpo de la estructura (como la declaración ilegal de variables en un bloque ejecutable), se adopta una estrategia de Modo Pánico. Se declara un Error Fatal y se detiene inmediatamente la ejecución del proceso de generación de código. Esto se debe a que la corrupción dentro del bloque ejecutable compromete la

integridad lógica de las instrucciones a iterar, haciendo inseguro cualquier intento de recuperación o generación de tercetos posteriores.

Tema 19: Asignación Múltiple

- **Análisis Sintáctico:** Se utiliza una pila auxiliar `pilaLadoDerecho` para acumular las expresiones o invocaciones detectadas antes de realizar la reducción de la regla de asignación. Esto permite procesar listas de longitud variable de manera dinámica. Para procesar asignaciones de la forma `a, b = 1, 2`, se utiliza una estrategia de acumulación diferida:
 - **Acumulación (`pilaLadoDerecho`):** En el archivo `Generador.java` se introdujo la pila `pilaLadoDerecho`. A medida que el parser reconoce los elementos a la derecha del operador de asignación (regla `lado_derecho_multiple`), éstos se apilan temporalmente en lugar de generar código inmediato. Esto permite manejar listas de longitud arbitraria antes de saber si coinciden con las variables de la izquierda.
 - **Procesamiento por Lotes:** Una vez leída toda la sentencia, se invoca el método `procesarAsignacionMultiple`. Este método desapila los operadores y los empareja posicionalmente con la lista de variables (`listaVariables`).
 - **Validación Semántica:** Se verifica que la cantidad de elementos a ambos lados coincida. Si hay discrepancia, se emite un Error Semántico (ajuste realizado para la reentrega, donde la discrepancia de dimensión es estricta) y se genera código solo para los pares válidos mínimos.
- **Mejora Reentrega (Robustez):** Se implementó una técnica de recuperación para tolerar el uso incorrecto del operador de asignación simple (`:=`) en contextos de asignación múltiple (donde se espera `=`).
 - **Acción:** Si el compilador detecta la estructura `a, b := 1, 2`, en lugar de fallar sintácticamente, aplica una técnica de borrado: elimina internamente el carácter dos puntos (`:`), asume que la operación es una asignación múltiple válida (`=`) y emite un Warning informativo ("El operador `':='` es para asignaciones simples..."). Esto permite que la compilación continúe y se genere el código correcto a pesar del error del usuario.

Tema 21: Retornos Múltiples

- **Análisis Semántico (Validación de Cantidad):** La lógica central se encuentra en el método auxiliar `procesarAsignacionMultiple`, que se invoca cuando se detecta una asignación a una lista de variables.
- **Validación de Correspondencia:** El compilador recupera la lista de `TiposRetorno` de la función invocada y compara su tamaño (`cantRetornos`) con la cantidad de variables a la izquierda (`cantIzquierda`).
- Se chequea rigurosamente la correspondencia entre la cantidad de variables receptoras (lado izquierdo) y la cantidad de valores retornados por la función (lado derecho). Se definieron dos comportamientos según el caso:

- **Caso LHS > RHS (Error Fatal):** Si hay más variables a la izquierda que valores retornados (ej. `a, b, c = func_retorna_2()`), se considera un Error Fatal. El compilador entra en Modo Pánico para esta sentencia, reporta el error semántico y aborta la generación de código para dicha instrucción, ya que es imposible satisfacer la asignación de variables faltantes sin comprometer la memoria.
- **Caso RHS > LHS (Warning):** Si la función retorna más valores de los que se asignan (ej. `a = func_retorna_2()`), el compilador adopta una estrategia permisiva. Se emite un Warning indicando que "se descartan los valores sobrantes" y se procede a generar el código de asignación sólo para las variables especificadas, ignorando los retornos adicionales.
- **Generación de Código:**
 - **Código Intermedio:** Se introduce el operador especial `GET_RET`. Se genera una secuencia lineal de asignaciones para extraer cada valor individual: `(GET_RET, [call_index], indice_parametro)`.
 - **Assembler:** Esto se traduce en la lectura de variables globales temporales (`_RET_VAL_0`, `_RET_VAL_1`, etc.), donde la función invocada depositó previamente sus resultados antes de retornar.

Tema 28: Expresiones Lambda

- **Análisis Sintáctico:** Las lambdas se definen gramaticalmente como funciones anidadas que pueden aparecer en expresiones.
 - **Salto de Definición:** Dado que el código de la lambda aparece en línea pero no debe ejecutarse en el momento de la definición, el parser emite inmediatamente un salto incondicional (BI) para "saltar" sobre el cuerpo de la función.
 - **Generación de Etiqueta:** Se marca el inicio del cuerpo con una etiqueta única (ej. Label15). El "valor" que retorna la expresión lambda al asignarse es, en realidad, una referencia simbólica a esta etiqueta (prefijo "L" + número de terceto), lo que permite tratar a la función como un puntero.
 - **Gestión de Ámbito:** Se abre un nuevo ámbito dinámico (lambda_ID) para aislar las variables y el parámetro de la lambda del resto del programa, cerrándolo al finalizar el bloque.
 - **Mejora Reentrega:** Se soporta recuperación de errores en la definición. Si la cabecera de la lambda es inválida, se crea un ámbito "dummy" (lambda_error) para poder analizar semánticamente el cuerpo sin detener la compilación.
- **Generación de Código:**
 1. Se emite un salto incondicional (BI) para saltar sobre el cuerpo de la lambda (que no se debe ejecutar al definirse).
 2. Se genera una etiqueta única y se compila el cuerpo.
 3. El "valor" de la lambda es la dirección de su etiqueta (FUNC_LABEL).
 4. Para la invocación, se utiliza el terceto CALL_LAMBDA, que realiza un salto indirecto a la dirección almacenada en la variable tipo lambda.

Tema 25: Semántica de Pasaje Copia-Resultado

- **Análisis Semántico:** La implementación del pasaje de parámetros por Copia-Resultado (Copy-Restore) se basa en una estrategia de "actualización diferida" gestionada por el invocador, asegurando que los cambios en los parámetros solo se reflejen en las variables originales una vez que la función ha retornado el control exitosamente.
 - **Copia de Entrada:** Se generan tercetos PARAM convencionales que copian el valor del argumento real (real.operando) a la variable local correspondiente de la función (nombreParametro). Esto aísla la ejecución de la función, que trabaja sobre sus propias copias locales en memoria.
- **Generación de Código:**
 - A diferencia del pasaje por referencia (punteros), se implementa la copia diferida.
 - Al llamar a la función, se copia el valor del parámetro real al formal (stack o registro).

- Al finalizar la función (RETURN), el código generado copia el valor final del parámetro formal de vuelta a la dirección de memoria del parámetro real, asegurando la actualización atómica sólo tras el éxito de la función.

Tema 31: Conversión Explícita (TOUI)

- **Análisis Semántico:** Se chequea que la conversión se aplique sobre una expresión compatible (float).
- La validación se realiza en dos niveles para asegurar la robustez del tipado:
 - **Validación de Conversión (gramatica.y):** En la regla `conversion_explicita`, se verifica que el operando pasado a `toui(...)` sea estrictamente de tipo float. Si se intenta convertir otro tipo (o si la expresión es inválida), se reporta un Error Semántico específico y se define el resultado como `uint` para permitir la continuidad del análisis.
 - **Restricción de Operaciones Mixtas (Generador.java):** Para dar sentido a la existencia de `toui`, se modificaron los métodos `chequearTipos` y `chequearAsignacion`. El compilador prohíbe explícitamente operar o asignar entre float y `uint` directamente, lanzando un error que instruye al usuario a utilizar la conversión explícita `toui` para resolver la incompatibilidad.
- **Generación de Código:** Se genera el terceto unario (TOUI, operando, _). En la etapa de Assembler, esto se traduce a la instrucción FISTP (Floating Point Integer Store Pop), que convierte el valor en el tope de la pila de la FPU a un entero y lo almacena en la variable destino, manejando la conversión de tipos a nivel de hardware.

Tema 32: Comentarios de 1 línea (@)

- **Análisis Léxico:** Se implementó el reconocimiento de este tipo de comentarios dentro del autómata finito (o mediante expresiones regulares, según usen). Al detectar el carácter `@`, el analizador léxico transiciona a un estado de consumo que ignora todos los caracteres subsiguientes hasta encontrar un salto de línea (`\n`) o el fin de archivo.
- **Impacto:** Estos caracteres son filtrados completamente en esta etapa y no generan tokens, siendo invisibles para el analizador sintáctico.

Generación de Código Assembler

Una vez obtenidos los tercetos del código intermedio, el sistema avanza hacia la emisión del código assembler compatible con MASM32. La ejecución de este archivo de salida permite realizar las pruebas funcionales dinámicas necesarias para verificar el cumplimiento de las reglas semánticas y de control de flujo.

Mecanismo General

Se utilizó el método de Variables Auxiliares. Para cada terceto que produce un resultado numérico, se reserva una variable de memoria (@auxN).

- **Enteros (uint):** Se utilizan los registros de propósito general de 32 bits (EAX, EBX, EDX).
- **Flotantes (float):** Se utiliza la pila del coprocesador matemático 80x87 (FLD, FSTP, FADD, etc.).

Mecanismo de generación de etiquetas de salto

Para resolver las bifurcaciones (saltos condicionales e incondicionales), se utilizó un esquema de etiquetado basado en los índices de los tercetos. Cada terceto generado tiene asociado una etiqueta potencial en el código Assembler.

Las etiquetas se construyen concatenando el prefijo fijo "Label" con el número de índice del terceto destino. Por ejemplo, si un terceto (BI, $\rightarrow$, [15]) indica un salto al terceto número 15, el generador producirá la instrucción JMP Label15. Antes de generar el código para el terceto 15, se inserta la etiqueta Label15: en el archivo de salida.

Estructura del Archivo Salida

El archivo generado consta de:

1. **Header:** Definiciones de arquitectura. En esta reentrega, se actualizó la directiva a .686 (anteriormente .386) para habilitar el conjunto de instrucciones extendido del Pentium Pro, optimizando el manejo de comparaciones de punto flotante. Se incluyen las librerías estándar (msvcrt, kernel32).
2. **.data:** Declaración de variables del usuario (con mangling), variables auxiliares, cadenas de texto y constantes para el control de errores en tiempo de ejecución (MaxFloatValue, mensajes de error).
3. **.code:** Etiqueta start: seguida de la traducción secuencial de tercetos.

Traducción Robusta de Operaciones Aritméticas

El compilador distingue dinámicamente el tipo de operación. Una mejora crítica en esta versión es la Inmunidad a Errores Previos: Antes de generar código para un terceto, el `GeneradorAssembler` verifica si el terceto está marcado como `error_tipo` (producto de la recuperación de errores semánticos). Si es así, omite la generación de instrucciones para ese terceto específico, insertando un comentario en el código destino. Esto evita que errores de compilación recuperados corrompan el ejecutable final o rompan el ensamblador.

Soporte para Lambdas

Se implementaron instrucciones específicas para manejar el flujo de funciones anónimas:

- **Pasaje de Parámetros (`PARAM_LAMBDA`):** A diferencia de las funciones normales que usan la pila, las lambdas utilizan el registro `ECX` (para enteros) o la variable global `_LAMBDA_ARG_FLOAT` (para flotantes) para un pasaje de argumentos rápido y directo.
- **Recepción (`DEF_PARAM`):** Al inicio de la lambda, se mueve el valor desde `ECX` o la global hacia la variable local del parámetro.
- **Invocación (`CALL_LAMBDA`):** Se realiza un `CALL` indirecto a la dirección de memoria almacenada en la variable de tipo lambda (`CALL EAX`).

Controles en Tiempo de Ejecución

Se incorporaron chequeos automáticos alrededor de las instrucciones aritméticas. Si se detecta una violación, se invoca a `ExitProcess` con un código de error.

b) Overflow en Sumas de Enteros

El tipo `uint` tiene un límite de 16 bits (65535), pero se opera en registros de 32 bits. Se verifica que el resultado no exceda este límite explícitamente.

- **Implementación:** Después de un `ADD EAX, op2`, se compara `EAX` con 65535.

```
codigo.append("ADD EAX, ").append(op2).append("\n");
codigo.append("CMP EAX, 65535\n");
codigo.append("JA ErrorOverflow\n");
```

e) Overflow en Productos de Datos de Punto Flotante

Para las operaciones flotantes, se verifica que el valor absoluto del resultado no exceda el máximo valor representable definido en la consigna.

- **Implementación:** Se definió `MaxFloatValue` dd 2139095039 en la sección de datos. Tras una operación (ej. `FMUL`), se compara el resultado.

```
codigo.append("FMUL\n");
codigo.append("FLD ST(0)\n");
codigo.append("FABS\n");
codigo.append("FCOMP MaxFloatValue\n");
codigo.append("FSTSW AX\n");
codigo.append("SAHF\n");
codigo.append("JA ErrorOverflow\n");
```

f) Resultados Negativos en Restas de Enteros sin Signo

Dado que el tipo `uint` no admite valores negativos, una resta $A - B$ donde $B > A$ debe producir un error en tiempo de ejecución (underflow).

- **Implementación:** Se utiliza el flag de acarreo (Carry Flag) que se activa si la resta requiere un "préstamo" (lo que indica resultado negativo en aritmética sin signo).

```
codigo.append("SUB EAX, ").append(op2).append("\n");
codigo.append("JC ErrorRestaNegativa\n");
```

Manejo de Comparaciones

Para las comparaciones ($<$, $>$, $=$), se utilizó el registro de estado y las instrucciones `SETxx`. Por ejemplo, para $<$ (menor): Se realiza `CMP` (enteros) o `FCOMIP` (flotantes), y luego `SETB AL` (Set if Below), moviendo el resultado (0 o 1) a la variable auxiliar.

Almacenamiento y Gestión de Variables Auxiliares

Las variables auxiliares se generan dinámicamente durante la creación de los tercetos y se almacenan directamente en la Tabla de Símbolos.

Como se puede observar en el archivo `salida_parser.txt`, las variables auxiliares aparecen listadas junto con las variables del usuario, identificadas con el lexema `@auxN` (donde N es el índice del terceto) y el atributo `Uso: variable_auxiliar`.

Siguiendo los principios de diseño de compiladores, las variables temporales o auxiliares generadas para la representación de código intermedio deben ser gestionadas formalmente por la Tabla de Símbolos por las siguientes razones:

- **Gestión de Memoria Unificada:** El generador de código final necesita reservar espacio de memoria estática para todas las entidades que almacenan valores, no solo las declaradas por el usuario. Al incluirlas en la Tabla de Símbolos, el compilador puede iterar sobre una única estructura para generar las directivas de datos de manera uniforme.
- **Resolución de Direcciones y Alcance:** Al igual que las variables de usuario, las auxiliares deben poseer un ámbito para evitar conflictos de nombres y asegurar que se accede a la dirección de memoria correcta durante la ejecución. Esto se evidencia mediante el Name Mangling aplicado también a las auxiliares (ej. @aux189:MAIN).
- **Consistencia de Atributos:** Permite asociar metadatos esenciales, como el tipo de dato que almacenan temporalmente, facilitando comprobaciones o conversiones posteriores sin requerir estructuras de datos paralelas y redundantes.

Decisión de Diseño en Generación de Código Assembler

Respecto a la eficiencia del código generado, el equipo es consciente de que la estrategia actual, basada en el uso de variables auxiliares en memoria para cada operación intermedia, conlleva una sobrecarga de accesos a la RAM en comparación con técnicas de asignación de registros (*Register Allocation*). Aunque reconocemos que maximizar la permanencia de valores en los registros de la CPU optimizaría el tiempo de ejecución, ante la ausencia de especificaciones estrictas sobre optimización en la consigna, se priorizó la robustez y la corrección de la traducción. Esta decisión de diseño garantiza la integridad del flujo de datos y simplifica la depuración, cumpliendo con los objetivos académicos de la etapa sin introducir la complejidad de algoritmos avanzados de gestión de registros.

Robustez y Recuperación de Errores

En el marco de la reentrega del proyecto, se ha realizado una refactorización profunda de los módulos de Análisis Léxico y Sintáctico. La diferencia fundamental entre la entrega anterior y la actual reside en el cambio de un enfoque estricto a uno robusto. Mientras que el compilador anterior detenía su ejecución ante el primer error léxico o sintáctico, la nueva versión implementa estrategias de recuperación de errores (error recovery) y corrección de intención (intent correction). Esto permite analizar la totalidad del archivo fuente en una sola pasada, reportando múltiples errores y generando código ejecutable siempre que sea seguro hacerlo.

A continuación, se detallan las técnicas de recuperación de errores implementadas y las modificaciones estructurales realizadas.

Analizador Léxico: Inferencia y Autocorrección

El cambio más técnico y profundo ocurre en el Autómata Finito y su implementación en la matriz de transición.

	"a"	"b"	"c"	"d"	"e"	"f"	"g"	"h"	"i"	"j"	"k"	"l"	"m"	"n"	"o"	"p"	"q"	"r"	"s"	"t"	"u"	"v"	"w"	"x"	"y"	"z"	"aa"	"ab"	"ac"	"ad"	"ae"	"af"	"ag"	"ah"	"ai"	"aj"	"ak"	"al"	"am"	"an"	"ao"	"ap"	"aq"	"ar"	"as"	"at"	"au"	"av"	"aw"	"ax"	"ay"	"az"	"ba"	"bb"	"bc"	"bd"	"be"	"bf"	"bg"	"bh"	"bi"	"bj"	"bk"	"bl"	"bm"	"bn"	"bo"	"bp"	"bq"	"br"	"bs"	"bt"	"bu"	"bv"	"bw"	"bx"	"by"	"bz"	"ca"	"cb"	"cc"	"cd"	"ce"	"cf"	"cg"	"ch"	"ci"	"cj"	"ck"	"cl"	"cm"	"cn"	"co"	"cp"	"cq"	"cr"	"cs"	"ct"	"cu"	"cv"	"cw"	"cx"	"cy"	"cz"	"da"	"db"	"dc"	"dd"	"de"	"df"	"dg"	"dh"	"di"	"dj"	"dk"	"dl"	"dm"	"dn"	"do"	"dp"	"dq"	"dr"	"ds"	"dt"	"du"	"dv"	"dw"	"dx"	"dy"	"dz"	"ea"	"eb"	"ec"	"ed"	"ee"	"ef"	"eg"	"eh"	"ei"	"ej"	"ek"	"el"	"em"	"en"	"eo"	"ep"	"eq"	"er"	"es"	"et"	"eu"	"ev"	"ew"	"ex"	"ey"	"ez"	"fa"	"fb"	"fc"	"fd"	"fe"	"ff"	"fg"	"fh"	"fi"	"fj"	"fk"	"fl"	"fm"	"fn"	"fo"	"fp"	"fq"	"fr"	"fs"	"ft"	"fu"	"fv"	"fw"	"fx"	"fy"	"fz"	"ga"	"gb"	"gc"	"gd"	"ge"	"gf"	"gg"	"gh"	"gi"	"gj"	"gk"	"gl"	"gm"	"gn"	"go"	"gp"	"gq"	"gr"	"gs"	"gt"	"gu"	"gv"	"gw"	"gx"	"gy"	"gz"	"ha"	"hb"	"hc"	"hd"	"he"	"hf"	"hg"	"hh"	"hi"	"hj"	"hk"	"hl"	"hm"	"hn"	"ho"	"hp"	"hq"	"hr"	"hs"	"ht"	"hu"	"hv"	"hw"	"hx"	"hy"	"hz"	"ia"	"ib"	"ic"	"id"	"ie"	"if"	"ig"	"ih"	"ii"	"ij"	"ik"	"il"	"im"	"in"	"io"	"ip"	"iq"	"ir"	"is"	"it"	"iu"	"iv"	"iw"	"ix"	"iy"	"iz"	"ja"	"jb"	"jc"	"jd"	"je"	"jf"	"jg"	"jh"	"ji"	"jj"	"jk"	"jl"	"jm"	"jn"	"jo"	"jp"	"jq"	"jr"	"js"	"jt"	"ju"	"jv"	"jw"	"jx"	"jy"	"jz"	"ka"	"kb"	"kc"	"kd"	"ke"	"kf"	"kg"	"kh"	"ki"	"kj"	"kk"	"kl"	"km"	"kn"	"ko"	"kp"	"kq"	"kr"	"ks"	"kt"	"ku"	"kv"	"kw"	"kx"	"ky"	"kz"	"la"	"lb"	"lc"	"ld"	"le"	"lf"	"lg"	"lh"	"li"	"lj"	"lk"	"ll"	"lm"	"ln"	"lo"	"lp"	"lq"	"lr"	"ls"	"lt"	"lu"	"lv"	"lw"	"lx"	"ly"	"lz"	"ma"	"mb"	"mc"	"md"	"me"	"mf"	"mg"	"mh"	"mi"	"mj"	"mk"	"ml"	"mm"	"mn"	"mo"	"mp"	"mq"	"mr"	"ms"	"mt"	"mu"	"mv"	"mw"	"mx"	"my"	"mz"	"na"	"nb"	"nc"	"nd"	"ne"	"nf"	"ng"	"nh"	"ni"	"nj"	"nk"	"nl"	"nm"	"nn"	"no"	"np"	"nq"	"nr"	"ns"	"nt"	"nu"	"nv"	"nw"	"nx"	"ny"	"nz"	"oa"	"ob"	"oc"	"od"	"oe"	"of"	"og"	"oh"	"oi"	"oj"	"ok"	"ol"	"om"	"on"	"oo"	"op"	"oq"	"or"	"os"	"ot"	"ou"	"ov"	"ow"	"ox"	"oy"	"oz"	"pa"	"pb"	"pc"	"pd"	"pe"	"pf"	"pg"	"ph"	"pi"	"pj"	"pk"	"pl"	"pm"	"pn"	"po"	"pp"	"pq"	"pr"	"ps"	"pt"	"pu"	"pv"	"pw"	"px"	"py"	"pz"	"qa"	"qb"	"qc"	"qd"	"qe"	"qf"	"qg"	"qh"	"qi"	"qj"	"qk"	"ql"	"qm"	"qn"	"qo"	"qp"	"qq"	"qr"	"qs"	"qt"	"qu"	"qv"	"qw"	"qx"	"qy"	"qz"	"ra"	"rb"	"rc"	"rd"	"re"	"rf"	"rg"	"rh"	"ri"	"rj"	"rk"	"rl"	"rm"	"rn"	"ro"	"rp"	"rq"	"rr"	"rs"	"rt"	"ru"	"rv"	"rw"	"rx"	"ry"	"rz"	"sa"	"sb"	"sc"	"sd"	"se"	"sf"	"sg"	"sh"	"si"	"sj"	"sk"	"sl"	"sm"	"sn"	"so"	"sp"	"sq"	"sr"	"ss"	"st"	"su"	"sv"	"sw"	"sx"	"sy"	"sz"	"ta"	"tb"	"tc"	"td"	"te"	"tf"	"tg"	"th"	"ti"	"tj"	"tk"	"tl"	"tm"	"tn"	"to"	"tp"	"tq"	"tr"	"ts"	"tt"	"tu"	"tv"	"tw"	"tx"	"ty"	"tz"	"ua"	"ub"	"uc"	"ud"	"ue"	"uf"	"ug"	"uh"	"ui"	"uj"	"uk"	"ul"	"um"	"un"	"uo"	"up"	"uq"	"ur"	"us"	"ut"	"uu"	"uv"	"uw"	"ux"	"uy"	"uz"	"va"	"vb"	"vc"	"vd"	"ve"	"vf"	"vg"	"vh"	"vi"	"vj"	"vk"	"vl"	"vm"	"vn"	"vo"	"vp"	"vq"	"vr"	"vs"	"vt"	"vu"	"vv"	"vw"	"vx"	"vy"	"vz"	"wa"	"wb"	"wc"	"wd"	"we"	"wf"	"wg"	"wh"	"wi"	"wj"	"wk"	"wl"	"wm"	"wn"	"wo"	"wp"	"wq"	"wr"	"ws"	"wt"	"wu"	"wv"	"ww"	"wx"	"wy"	"wz"	"xa"	"xb"	"xc"	"xd"	"xe"	"xf"	"xg"	"xh"	"xi"	"xj"	"xk"	"xl"	"xm"	"xn"	"xo"	"xp"	"xq"	"xr"	"xs"	"xt"	"xu"	"xv"	"xw"	"xx"	"xy"	"xz"	"ya"	"yb"	"yc"	"yd"	"ye"	"yf"	"yg"	"yh"	"yi"	"yj"	"yk"	"yl"	"ym"	"yn"	"yo"	"yp"	"yq"	"yr"	"ys"	"yt"	"yu"	"yv"	"yw"	"zx"	"zy"	"zz"
0	F	1	3	2	3	4	5	5	6	7	16	9	ERROR	ERROR	0	0	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F																																																																																																	

[Acceso a Tabla de Transición de Estados](#)

	"a"	"b"	"c"	"d"	"e"	"f"	"g"	"h"	"i"	"j"	"k"	"l"	"m"	"n"	"o"	"p"	"q"	"r"	"s"	"t"	"u"	"v"	"w"	"x"	"y"	"z"	"aa"	"ab"	"ac"	"ad"	"ae"	"af"	"ag"	"ah"	"ai"	"aj"	"ak"	"al"	"am"	"an"	"ao"	"ap"	"aq"	"ar"	"as"	"at"	"au"	"av"	"aw"	"ax"	"ay"	"az"	"ba"	"bb"	"bc"	"bd"	"be"	"bf"	"bg"	"bh"	"bi"	"bj"	"bk"	"bl"	"bm"	"bn"	"bo"	"bp"	"bq"	"br"	"bs"	"bt"	"bu"	"bv"	"bw"	"bx"	"by"	"bz"	"ca"	"cb"	"cc"	"cd"	"ce"	"cf"	"cg"	"ch"	"ci"	"cj"	"ck"	"cl"	"cm"	"cn"	"co"	"cp"	"cq"	"cr"	"cs"	"ct"	"cu"	"cv"	"cw"	"cx"	"cy"	"cz"	"da"	"db"	"dc"	"dd"	"de"	"df"	"dg"	"dh"	"di"	"dj"	"dk"	"dl"	"dm"	"dn"	"do"	"dp"	"dq"	"dr"	"ds"	"dt"	"du"	"dv"	"dw"	"dx"	"dy"	"dz"	"ea"	"eb"	"ec"	"ed"	"ee"	"ef"	"eg"	"eh"	"ei"	"ej"	"ek"	"el"	"em"	"en"	"eo"	"ep"	"eq"	"er"	"es"	"et"	"eu"	"ev"	"ew"	"ex"	"ey"	"ez"	"fa"	"fb"	"fc"	"fd"	"fe"	"ff"	"fg"	"fh"	"fi"	"fj"	"fk"	"fl"	"fm"	"fn"	"fo"	"fp"	"fq"	"fr"	"fs"	"ft"	"fu"	"fv"	"fw"	"fx"	"fy"	"fz"	"ga"	"gb"	"gc"	"gd"	"ge"	"gf"	"gg"	"gh"	"gi"	"gj"	"gk"	"gl"	"gm"	"gn"	"go"	"gp"	"gq"	"gr"	"gs"	"gt"	"gu"	"gv"	"gw"	"gx"	"gy"	"gz"	"ha"	"hb"	"hc"	"hd"	"he"	"hf"	"hg"	"hh"	"hi"	"hj"	"hk"	"hl"	"hm"	"hn"	"ho"	"hp"	"hq"	"hr"	"hs"	"ht"	"hu"	"hv"	"hw"	"hx"	"hy"	"hz"	"ia"	"ib"	"ic"	"id"	"ie"	"if"	"ig"	"ih"	"ii"	"ij"	"ik"	"il"	"im"	"in"	"io"	"ip"	"iq"	"ir"	"is"	"it"	"iu"	"iv"	"iw"	"ix"	"iy"	"iz"	"ja"	"jb"	"jc"	"jd"	"je"	"jf"	"jg"	"jh"	"ji"	"jj"	"jk"	"jl"	"jm"	"jn"	"jo"	"jp"	"jq"	"jr"	"js"	"jt"	"ju"	"jv"	"jw"	"jx"	"jy"	"jz"	"ka"	"kb"	"kc"	"kd"	"ke"	"kf"	"kg"	"kh"	"ki"	"kj"	"kk"	"kl"	"km"	"kn"	"ko"	"kp"	"kq"	"kr"	"ks"	"kt"	"ku"	"kv"	"kw"	"kx"	"ky"	"kz"	"la"	"lb"	"lc"	"ld"	"le"	"lf"	"lg"	"lh"	"li"	"lj"	"lk"	"ll"	"lm"	"ln"	"lo"	"lp"	"lq"	"lr"	"ls"	"lt"	"lu"	"lv"	"lw"	"lx"	"ly"	"lz"	"ma"	"mb"	"mc"	"md"	"me"	"mf"	"mg"	"mh"	"mi"	"mj"	"mk"	"ml"	"mm"	"mn"	"mo"	"mp"	"mq"	"mr"	"ms"	"mt"	"mu"	"mv"	"mw"	"mx"	"my"	"mz"	"na"	"nb"	"nc"	"nd"	"ne"	"nf"	"ng"	"nh"	"ni"	"nj"	"nk"	"nl"	"nm"	"nn"	"no"	"np"	"nq"	"nr"	"ns"	"nt"	"nu"	"nv"	"nw"	"nx"	"ny"	"nz"	"oa"	"ob"	"oc"	"od"	"oe"	"of"	"og"	"oh"	"oi"	"oj"	"ok"	"ol"	"om"	"on"	"oo"	"op"	"oq"	"or"	"os"	"ot"	"ou"	"ov"	"ow"	"ox"	"oy"	"oz"	"pa"	"pb"	"pc"	"pd"	"pe"	"pf"	"pg"	"ph"	"pi"	"pj"	"pk"	"pl"	"pm"	"pn"	"po"	"pp"	"pq"	"pr"	"ps"	"pt"	"pu"	"pv"	"pw"	"px"	"py"	"pz"	"qa"	"qb"	"qc"	"qd"	"qe"	"qf"	"qg"	"qh"	"qi"	"qj"	"qk"	"ql"	"qm"	"qn"	"qo"	"qp"	"qq"	"qr"	"qs"	"qt"	"qu"	"qv"	"qw"	"qx"	"qy"	"qz"	"ra"	"rb"	"rc"	"rd"	"re"	"rf"	"rg"	"rh"	"ri"	"rj"	"rk"	"rl"	"rm"	"rn"	"ro"	"rp"	"rq"	"rr"	"rs"	"rt"	"ru"	"rv"	"rw"	"rx"	"ry"	"rz"	"sa"	"sb"	"sc"	"sd"	"se"	"sf"	"sg"	"sh"	"si"	"sj"	"sk"	"sl"	"sm"	"sn"	"so"	"sp"	"sq"	"sr"	"ss"	"st"	"su"	"sv"	"sw"	"sx"	"sy"	"sz"	"ta"	"tb"	"tc"	"td"	"te"	"tf"	"tg"	"th"	"ti"	"tj"	"tk"	"tl"	"tm"	"tn"	"to"	"tp"	"tq"	"tr"	"ts"	"tt"	"tu"	"tv"	"tw"	"tx"	"ty"	"tz"	"ua"	"ub"	"uc"	"ud"	"ue"	"uf"	"ug"	"uh"	"ui"	"uj"	"uk"	"ul"	"um"	"un"	"uo"	"up"	"uq"	"ur"	"us"	"ut"	"uu"	"uv"	"uw"	"ux"	"uy"	"uz"	"va"	"vb"	"vc"	"vd"	"ve"	"vf"	"vg"	"vh"	"vi"	"vj"	"vk"	"vl"	"vm"	"vn"	"vo"	"vp"	"vq"	"vr"	"vs"	"vt"	"vu"	"vv"	"vw"	"vx"	"vy"	"vz"	"wa"	"wb"	"wc"	"wd"	"we"	"wf"	"wg"	"wh"	"wi"	"wj"	"wk"	"wl"	"wm"	"wn"	"wo"	"wp"	"wq"	"wr"	"ws"	"wt"	"wu"	"wv"	"ww"	"wx"	"wy"	"wz"	"xa"	"xb"	"xc"	"xd"	"xe"	"xf"	"xg"	"xh"	"xi"	"xj"	"xk"	"xl"	"xm"	"xn"	"xo"	"xp"	"xq"	"xr"	"xs"	"xt"	"xu"	"xv"	"xw"	"xx"	"xy"	"xz"	"ya"	"yb"	"yc"	"yd"	"ye"	"yf"	"yg"	"yh"	"yi"	"yj"	"yk"	"yl"	"ym"	"yn"	"yo"	"yp"	"yq"	"yr"	"ys"	"yt"	"yu"	"yv"	"yw"	"zx"	"zy"	"zz"
0	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as1	as																																																																																																																																																																																																																					

Acceso a Tabla de Acciones Semánticas

Modificación de la Matriz de Transición y Acciones Semánticas

Se ha rediseñado la matriz de estados y acciones semánticas (AnalizadorLexico.java y AccionSemantica.java) para manejar transiciones que anteriormente conducían a un estado de error inmediato.

- **Detección de Asignaciones Mal Formadas:** En la versión anterior, el estado correspondiente a la lectura del carácter dos puntos (:) (Estado 2) sólo aceptaba

un signo igual (=) subsiguiente. Cualquier otro caracter provocaba un error léxico fatal. En la nueva versión, se han implementado las siguientes correcciones automáticas mediante nuevas acciones semánticas (asC y asCE):

- **Caso “:+" o “:.”** : Si se detecta un carácter inválido tras los dos puntos, se ejecuta la *AccionSemanticaCorreccion*. Esta acción emite un Warning, fuerza el lexema a := y permite continuar el análisis, asumiendo que el usuario intentó realizar una asignación.
- **Caso “:=”**: Se introdujo la *AccionSemanticaCorreccionExtra* para consumir caracteres redundantes (como un segundo =) y normalizar el token a un operador de asignación válido ASIG.
- **Matriz de Transición**: Se ha añadido un estado de aceptación auxiliar (Estado 17) que permite finalizar el token corregido y retornar el control al parser de manera segura, evitando excepciones en tiempo de ejecución.

Validación Semántica de Sentencias RETURN

Con el objetivo de garantizar la integridad del flujo de datos y evitar comportamientos indefinidos en el código Assembler generado (como desbalanceo de la pila o registros con valores basura), se incorporó un control semántico estricto para verificar la existencia de sentencias de retorno en todas las funciones declaradas.

- **Problema Detectado**: En versiones anteriores, era posible declarar una función con tipo de retorno (ej. uint o float) que finalizaba su ejecución sin haber invocado explícitamente a RETURN. Esto generaba código ejecutable defectuoso donde la función invocadora leía datos inválidos.
- **Lógica de Implementación**: Se implementó un mecanismo de validación basado en pilas auxiliares dentro de la gramática (gramatica.y):
- **Control de Estado (pilaHuboRetorno)**: Al iniciar el análisis de una función, se apila un flag de estado inicializado en false.
- **Detección**: Cada vez que el parser reconoce una sentencia RETURN válida dentro del bloque, actualiza el flag en el tope de la pila a true.
- **Verificación al Cierre**: Al encontrar la llave de cierre “}” de la función, se desapila el estado. Si el valor es false, el compilador emite inmediatamente un Error Semántico (ej: "La funcion 'X' debe retornar un valor de tipo uint, anulando la generación de código para esa unidad).
- **Cobertura**: Este control aplica tanto para funciones de retorno simple como para funciones de retornos múltiples (Tema 21), asegurando que se cumpla la firma de la función antes de permitir la generación del ejecutable.

Precisión en el Reporte de Errores

Para mejorar la experiencia de depuración, se solucionó el problema de desfase de líneas causado por el lookahead del parser.

- **Tracking de Líneas (líneaAnterior vs líneaActual):** Se implementó un mecanismo en el AnalizadorLexico para recordar la ubicación del token previo. Esto asegura que los mensajes de error sintáctico apunten exactamente a la línea donde ocurre el fallo (por ejemplo, donde falta el punto y coma) y no a la línea siguiente, como ocurría en la versión anterior.

Acciones Semánticas: Saneamiento de Entrada

En esta nueva entrega, las acciones semánticas ahora funcionan como filtros de validación y limpieza de datos, superando su rol anterior de simples constructores de tokens.

Nuevas Clases de Acción (Corrección Léxica)

Se crearon clases específicas en AccionSemantica.java para manejar errores comunes de tipeo:

- **AccionSemanticaCorreccion (asC):**
 - **Función:** Intercepta errores como :+, :-, : (dos puntos espacio).
 - **Lógica:** Emite un Warning, reemplaza internamente el lexema corrupto por := y devuelve el token ASIG.
 - **Justificación:** Evita detener la compilación por errores triviales de digitación.
- **AccionSemanticaCorreccionExtra (asCE):**
 - **Función:** Maneja el error de "tecla repetida" (ej. :=:=).
 - **Lógica:** Consume el carácter excedente y normaliza el token a :=.

Manejo de Rangos Numéricos

Se ha modificado la lógica de validación de constantes numéricas para asegurar la generación de código Assembler válido, incluso ante desbordamientos en el código fuente.

- **Enteros Sin Signo (UINT):** Se mantiene la validación de rango (0 a 65535). Sin embargo, la detección de fuera de rango ahora permite al parser decidir la estrategia de recuperación en lugar de aceptar un token inválido silenciosamente.
- **Punto Flotante (FLOAT):** Anteriormente, un desbordamiento (overflow o underflow) generaba un error pero permitía el paso de valores inválidos a la tabla de símbolos. La nueva implementación aplica Clamping (saturación):

- **Overflow:** Si el valor supera el máximo permitido (3.40282347E+38), se emite un Warning y se reemplaza el valor por el máximo representable.
- **Underflow:** Si el valor es inferior al mínimo positivo (1.17549435E-38), se reemplaza por el mínimo representable. Esto garantiza que la etapa de generación de código nunca reciba valores numéricos que la arquitectura destino no pueda procesar.

Gramática: Sincronización y Recuperación

El archivo gramaticay incorporó el uso del token especial error de Yacc para manejar situaciones de pánico controladas.

Regla fin_sentencia (Inferencia de “;”)

- **Cambio:** Se creó una regla que deriva en ; o en vacío.
- **Lógica:** Si el parser encuentra el “;”, funciona normal. Si no lo encuentra, reduce por la regla vacía, emite un Warning (“Se asume punto y coma”) y continúa analizando la siguiente sentencia.
- **Justificación:** Es el error más común en programación. Permitir que el compilador continúe tras un “;” faltante es esencial para la usabilidad.

Estructuras de Control (IF / DO-WHILE)

- **Condiciones Inválidas:** Se agregaron reglas IF (error) y WHILE (error). Si la expresión condicional está mal formada (ej. paréntesis desbalanceados o tipos incompatibles), el parser descarta la condición corrupta pero mantiene la estructura del bloque, permitiendo validar el código dentro del cuerpo del IF o WHILE.
- **Cierre de Bloques:** Se implementó recuperación para la falta de ENDIF. Si el parser llega al final de un bloque lógico sin encontrar el token de cierre, lo inserta virtualmente, emite un aviso y cierra el ámbito en la Tabla de Símbolos para evitar errores de alcance en cascada.

Función Lambda

- **Problema Anterior:** Una definición de lambda incorrecta (ej. falta de parámetros) detenía el análisis, impidiendo verificar el código interno de la función.
- **Solución Actual:** Ante un error en la cabecera de la lambda, se ejecuta una acción de recuperación que crea un ámbito "dummy" (lambda_error...). Esto "engaña" al compilador haciéndole creer que está en una función válida, permitiéndole analizar y reportar errores dentro del cuerpo de la lambda.

Restricción Explícita de Declaraciones

- **Nuevo:** Se agregó la regla `declaracion_ilegal` en bloques ejecutables.
- **Justificación:** En lugar de lanzar un "Syntax Error" genérico cuando el usuario declara una variable (VAR) en medio del código, ahora se lanza un mensaje específico y educativo: "No se permiten declaraciones en bloques ejecutables", mejorando la claridad semántica.

Recuperación de Errores Sintácticos

Se han introducido reglas de error en la gramática (`gramatica.y`) para sincronizar el análisis tras un fallo y evitar el "modo pánico" que descartaba grandes porciones de código.

Inferencia de Fin de Sentencia

Se creó la regla `fin_sentencia` para gestionar la ausencia de puntos y coma (;).

- **Antes:** La ausencia de ; provocaba un Syntax Error inmediato.
- **Ahora:** La gramática acepta la ausencia del token ;. Si no se encuentra, se genera un Warning ("Falta punto y coma, se asume su existencia") y se continúa el análisis de la siguiente sentencia. Esto permite detectar múltiples errores en una sola compilación.

Recuperación en Estructuras de Control (IF / DO-WHILE)

Se robusteció el análisis de estructuras de control para tolerar errores en condiciones y bloques:

- **Corrección Semántica en Condiciones:** Si el usuario utiliza el operador de asignación `:=` dentro de una condición IF (ej. `if (a := b)`), el parser ya no falla. En su lugar, emite un Warning indicando el uso incorrecto y transforma internamente la operación a una comparación de igualdad (`==`), corrigiendo la lógica del programa al vuelo.

- **Cierre de Bloques (ENDIF):** Se agregaron reglas específicas para detectar la omisión de la palabra reservada ENDIF. El compilador es capaz de inferir el cierre del bloque IF al encontrar el final de la estructura o el inicio de otra, emitiendo la advertencia correspondiente sin detener la compilación.

Estabilidad en Expresiones Lambda

Se han añadido reglas para manejar declaraciones de funciones lambda defectuosas (por ejemplo, con listas de parámetros vacías o mal formadas). El compilador genera una estructura "dummy" interna para la lambda errónea, lo que permite validar semánticamente el cuerpo de la función (sentencias internas) aunque la cabecera sea incorrecta.

Visualización Extendida y Reporte de la Tabla de Símbolos

Como parte de las herramientas de diagnóstico y validación del compilador, se realizó una actualización significativa en el módulo de reporte de la Tabla de Símbolos. Esta mejora fue diseñada específicamente para facilitar la verificación de los atributos semánticos complejos introducidos en esta etapa, particularmente aquellos relacionados con el pasaje de parámetros y la gestión de tipos.

Objetivo: Proporcionar una visión transparente del estado interno del compilador, permitiendo a los desarrolladores corroborar que las reglas de inferencia y asignación de atributos (como el tipo de dato y el mecanismo de pasaje) se están aplicando correctamente antes de la generación de código.

Implementación Técnica: Se refactorizó el método `imprimirTablaSimbolos()` dentro de la clase `AnalizadorLexico`. El reporte ahora genera una salida tabular estructurada que recorre todos los ámbitos abiertos y cerrados, mostrando no solo el lexema y su ámbito, sino también sus metadatos críticos.

Columna "Pasaje": Se incorporó explícitamente esta columna para distinguir el mecanismo de paso de parámetros asignado a cada identificador. Esto es fundamental para validar el Tema 25 (Copia-Resultado), permitiendo diferenciar visualmente entre parámetros por copia-valor (`default_cv`) y parámetros por copia-resultado (`cr_se` o `cr_le`) directamente en la salida de consola.

Detalle de Atributos Reportados: Además del mecanismo de pasaje, el reporte expone:

- **Uso:** Clasifica la entidad (variable, función, parámetro, variable auxiliar), esencial para confirmar que el Name Mangling y la resolución de nombres funcionan según lo esperado.
- **Tipo:** Muestra el tipo de dato inferido o declarado (`uint`, `float`, `lambda`, `multiple`), facilitando la detección de errores de tipado antes de la etapa de generación de Assembler.

- **Estructura:** La visualización respeta la jerarquía de ámbitos, permitiendo observar cómo se anidan las variables y funciones (incluyendo las lambdas y sus ámbitos anónimos).

Conclusión

La finalización de estas etapas marca el cierre exitoso del desarrollo del compilador, logrando un producto de software capaz de transformar código fuente de alto nivel en un ejecutable funcional y eficiente. La implementación de la generación de código intermedio mediante Tercetos demostró ser una estrategia vital para desacoplar la complejidad del análisis sintáctico de la especificidad del lenguaje máquina, facilitando particularmente la resolución de estructuras anidadas y el manejo de ámbitos en las expresiones lambda.

Es imperativo destacar que el éxito de esta reentrega se debe, en gran medida, a la ampliación significativa de la cobertura de pruebas. Gracias a la incorporación de casos de prueba más extensivos, fue posible aislar comportamientos anómalos y detectar errores sutiles en la gestión de registros y en la lógica de control de flujo que no eran evidentes en pruebas simplificadas. Esta rigurosidad en la validación nos permitió identificar y resolver la totalidad de los problemas hallados, asegurando que características críticas —como el Name Mangling, la coerción de tipos y las verificaciones de overflow en tiempo de ejecución— operen con la estabilidad esperada.

En definitiva, este proyecto no solo ha permitido aplicar los conceptos teóricos de la construcción de compiladores, sino que también ha reafirmado la importancia del testing exhaustivo como una fase inseparable del desarrollo de software robusto.